BCSE498J Project-II - Capstone Project

DEEPSECURE - AI POWERED IDS FOR REAL-TIME NETWORK TRAFFIC

22BCI0196 RAMANSH VASANIA

22BCI0228 RAVIPATI ANAGHA

Under the Supervision of

Dr. Jim Solomon Raja D

Assistant Professor Sr. Grade 1

School of Computer Science and Engineering (SCOPE)

B.Tech.

in

COMPUTER SCIENCE AND ENGINEERING (WITH SPECIALIZATION IN INFORMATION SECURITY)

School of Computer Science and Engineering (SCOPE)

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

February 2026

ABSTRACT

This project presents DeepSecure, an AI-powered Intrusion Detection System (IDS) designed to monitor and analyze network traffic for identifying suspicious and malicious activities in near real-time. With the rapid growth of digital communication and interconnected systems, modern networks face increasing threats such as unauthorized access, malware, and evolving cyberattacks. Traditional intrusion detection systems primarily rely on static rules and predefined signatures, which limits their effectiveness against new and previously unseen attack patterns. To overcome these limitations, this project explores the use of artificial intelligence to build a more adaptive and intelligent network security solution.

The proposed system learns patterns of normal and abnormal network behavior using machine learning techniques trained on standard network traffic datasets. By extracting and analysing key features from network traffic, the system classifies incoming data as either normal or suspicious. This learning-based approach enables the detection of both known attack types and anomalous behaviours that may indicate potential security threats. The system is designed to simulate real-time traffic analysis, allowing timely detection and alert generation.

In addition to detection, the project emphasizes usability by presenting intrusion alerts and traffic status through a simple and understandable interface. This visualization helps users or network administrators quickly interpret security events and take appropriate action. The results demonstrate that AI-based intrusion detection can improve detection accuracy, adaptability, and early threat identification when compared to traditional rule-based systems.

Overall, this project highlights the effectiveness of artificial intelligence in enhancing network security by strengthening early warning mechanisms and providing a practical, scalable approach to intrusion detection in modern digital environments.

TABLE OF CONTENTS

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

The rapid growth of computer networks, cloud-based services, and internet-enabled applications has significantly increased the volume and complexity of network traffic in modern digital environments. While these advancements have enabled efficient communication and data sharing, they have also expanded the attack surface for cyber threats such as unauthorized access, malware, denial-of-service attacks, and zero-day exploits. As a result, ensuring network security has become a critical concern for organizations across various domains.

Intrusion Detection Systems (IDS) play a vital role in monitoring network traffic and identifying suspicious or malicious activities. Traditional IDS approaches primarily rely on rule-based or signature-based mechanisms, which are effective in detecting known attack patterns but often fail to identify new or evolving threats. To overcome these limitations, recent research has increasingly focused on applying artificial intelligence (AI), particularly machine learning (ML) and deep learning (DL) techniques, to improve intrusion detection accuracy and adaptability.

AI-based IDS models learn patterns from historical network traffic data and classify incoming traffic as normal or malicious. While these approaches have shown promising results in controlled experimental settings, many existing solutions emphasize high detection accuracy using complex architectures and advanced datasets. However, such systems often face challenges related to real-time applicability, interpretability, computational complexity, and practical deployment in real-world environments.

1.2 MOTIVATION

Despite significant progress in AI-driven intrusion detection, several critical limitations remain in existing research and implementations. A major concern is the growing reliance on highly complex deep learning architectures, which, although accurate, require substantial computational resources and limit real-world deployability. This creates a gap between research-oriented IDS models and practically usable security solutions.

Additionally, most existing IDS studies focus primarily on improving classification accuracy through offline evaluation, with limited emphasis on real-time or near real-time detection behavior. In practical scenarios, timely detection and alert generation are as important as accuracy. Furthermore, many AI-based IDS solutions provide limited

interpretability, making it difficult for network administrators to understand alerts and take informed actions.

Another key motivation arises from the excessive dependence on deep learning techniques for detecting zero-day attacks, often overlooking simpler machine learning approaches that can effectively identify anomalous behavior. Existing systems also tend to focus on individual components of intrusion detection, such as classification, without integrating detection, alerting, and visualization into a unified pipeline. Moreover, many approaches rely heavily on specific benchmark datasets without demonstrating adaptability to different network environments.

These challenges highlight the need for a lightweight, interpretable, and practical AI-based IDS that balances detection performance with real-time feasibility and usability.

1.3 SCOPE OF THE PROJECT

The scope of this project is to design and develop DeepSecure, an AI-powered Intrusion Detection System that addresses the identified research gaps by adopting a practical and balanced approach. The project focuses on implementing a machine learning-based IDS capable of analyzing network traffic and identifying suspicious activities in near real-time, without relying on overly complex deep learning architectures.

The system emphasizes real-time traffic simulation, efficient feature extraction, and accurate classification of normal and malicious network behavior. To improve usability and interpretability, the project integrates a simple visualization mechanism that presents alerts and traffic status in a clear and understandable manner for users or network administrators. The proposed architecture follows a unified pipeline that combines traffic analysis, intrusion detection, alert generation, and visualization within a single framework.

Furthermore, the project is designed to be adaptable to standard intrusion detection datasets, allowing retraining and extension to different network environments. While the current implementation focuses on simulated real-time detection, the system architecture supports future integration with live network traffic sources. Overall, this project aims to demonstrate a scalable, interpretable, and deployable AI-based IDS that bridges the gap between academic research and practical network security solutions.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

This section presents a comprehensive review of existing research on Intrusion Detection Systems (IDS), focusing on the evolution of detection techniques, the increasing use of artificial intelligence, and the persistent limitations identified across current approaches. Several studies in the literature have examined IDS from different perspectives, including detection accuracy, learning methodologies, dataset usage, and deployment feasibility. By synthesizing findings from a large body of work, this review highlights key trends and unresolved challenges that motivate the proposed project.

2.1.1 TRADITIONAL INTRUSION DETECTION SYSTEMS

Early intrusion detection systems were primarily based on signature-based and rule-based detection mechanisms. As discussed in multiple foundational studies, these systems relied on predefined attack signatures or expert-defined rules to identify malicious activities within network traffic. Such approaches were effective in detecting known attacks and offered low computational overhead, making them suitable for early network environments.

However, several researchers have reported inherent limitations of traditional IDS. Since signatures must be manually updated, these systems fail to detect novel, polymorphic, and zero-day attacks. Prior studies have also pointed out scalability issues in rule-based IDS, where increasing network traffic volume and protocol diversity lead to higher false-negative rates. These shortcomings highlighted the need for adaptive and learning-based intrusion detection techniques capable of handling evolving threat landscapes.

2.1.2 MACHINE LEARNING-BASED INTRUSION DETECTION SYSTEMS

With the advancement of artificial intelligence, machine learning-based IDS emerged as a data-driven alternative to traditional approaches. As reported in numerous studies, supervised learning algorithms such as decision trees, support vector machines, k-nearest neighbors, artificial neural networks, and ensemble classifiers have been widely adopted for intrusion detection tasks.

Most existing research follows a common methodology involving preprocessing of network traffic data, feature extraction, model training, and performance evaluation using benchmark datasets such as KDD Cup 99, NSL-KDD, CICIDS, and UNSW-NB15. Several comparative studies have demonstrated that ML-based IDS significantly

improve detection accuracy and reduce false-positive rates when compared to signature-based systems, particularly for well-defined attack categories like denial-of-service and probing attacks.

The importance of feature selection has been emphasized across multiple research works. Reducing feature dimensionality has been shown to improve classification efficiency and lower computational overhead. Ensemble learning techniques, especially Random Forest models, are frequently reported to offer improved generalization by combining multiple decision boundaries. Despite these advantages, many studies acknowledge that ML-based IDS are often evaluated offline and lack real-time detection capabilities. Additionally, class imbalance and limited adaptability to unseen traffic patterns remain unresolved challenges.

2.1.3 DEEP LEARNING-BASED INTRUSION DETECTION SYSTEMS

Recent research has increasingly focused on deep learning techniques for intrusion detection due to their ability to automatically learn complex feature representations. Several studies have explored convolutional neural networks to capture spatial relationships in network traffic features, while recurrent neural networks and long short-term memory models have been employed to model temporal dependencies in sequential data.

Autoencoders and generative adversarial networks have also been proposed for anomaly detection and zero-day attack identification by learning representations of normal network behavior. According to findings reported in multiple studies, deep learning-based IDS often outperform traditional machine learning models in terms of detection accuracy, especially in multi-class intrusion detection scenarios.

However, the literature consistently highlights practical limitations of deep learning-based IDS. Many researchers note that these models require large volumes of labeled data, extensive training time, and significant computational resources, which restrict their deployment in real-world environments. Furthermore, deep learning models are frequently criticized for their black-box nature, offering limited interpretability of detection decisions and reducing their usefulness for network administrators.

2.1.4 HYBRID AND ENSEMBLE INTRUSION DETECTION APPROACHES

To address the limitations of individual machine learning and deep learning models, several studies have proposed hybrid and ensemble intrusion detection approaches. These systems combine multiple classifiers, integrate feature fusion strategies, or employ optimization algorithms to enhance detection performance.

Research findings indicate that hybrid IDS architectures can achieve higher accuracy and improved robustness across benchmark datasets. However, many authors also note that these performance gains often come at the cost of increased architectural complexity and computational overhead. A recurring observation in the literature is that

most hybrid IDS solutions focus heavily on classification accuracy while overlooking practical aspects such as real-time detection, interpretability, and ease of deployment.

2.1.5 REAL-TIME DETECTION AND PRACTICAL DEPLOYMENT CHALLENGES

Although real-time intrusion detection is frequently stated as an objective, a majority of existing studies evaluate IDS models using offline datasets and batch processing techniques. Several researchers have highlighted the lack of real-time traffic analysis and continuous monitoring in current IDS implementations. Real-time deployment introduces additional challenges, including detection latency, scalability, and resource efficiency, which are often insufficiently addressed in experimental evaluations.

Another challenge repeatedly discussed in the literature is the lack of user-centric design in IDS solutions. Many systems generate alerts without meaningful explanations, making it difficult for administrators to interpret results and take timely action. Additionally, heavy reliance on specific benchmark datasets has raised concerns about the adaptability and generalization of IDS models when applied to different network environments.

2.1.6 SUMMARY OF LITERATURE AND RESEARCH DIRECTION

The reviewed literature demonstrates significant progress in the application of artificial intelligence to intrusion detection systems. Researchers consistently report improvements in detection accuracy and adaptability through machine learning and deep learning techniques. However, existing studies often prioritize model complexity and offline performance over deployability, real-time detection behavior, interpretability, and adaptability.

These findings reveal a clear research gap and emphasize the need for a lightweight, interpretable, and practical intrusion detection system. The proposed project is directly motivated by these observations and aims to address the identified limitations by developing an AI-based IDS with near real-time detection and integrated visualization capabilities.

2.2 RESEARCH GAP

Despite extensive research on intrusion detection systems using artificial intelligence, several critical gaps remain unresolved in existing studies. These gaps indicate a clear disconnect between academic research outcomes and the practical requirements of real-world network security systems.

Gap 1: Overly Complex IDS Architectures Limiting Real-World Deployability

A significant portion of recent IDS research focuses on highly complex deep learning architectures, including hybrid CNN–LSTM models, attention mechanisms, and generative adversarial networks. While these models often achieve high detection accuracy in experimental settings, they require substantial computational resources, large memory capacity, and long training times. Such requirements limit their feasibility for deployment in real-world or resource-constrained environments, such as small organizations or edge networks. There is a lack of research emphasizing lightweight and efficient IDS architectures that balance detection performance with practical deployability.

Gap 2: Limited Focus on Real-Time Detection Behavior

Most IDS studies evaluate model performance using offline datasets and batch processing techniques, primarily emphasizing accuracy, precision, and recall. However, in real-world scenarios, the ability to detect intrusions in real time or near real time is equally important. Detection latency, alert responsiveness, and continuous monitoring are often overlooked. This creates a gap between theoretical performance evaluation and operational IDS requirements, where timely detection and response are critical for minimizing damage.

Gap 3: Poor Interpretability of AI-Based IDS Outputs

Many AI-based IDS solutions function as black-box models, providing limited insight into how detection decisions are made. Alerts are often presented as raw classifications without meaningful explanations. This lack of interpretability makes it difficult for network administrators to understand the nature of detected threats, assess their severity, and take appropriate action. Existing research places limited emphasis on presenting IDS outputs in a human-understandable and actionable manner.

Gap 4: Excessive Dependence on Deep Learning for Zero-Day Detection

Several studies assume that deep learning techniques are essential for detecting zero-day or previously unseen attacks. While deep learning models are powerful, this assumption often overlooks the potential of traditional machine learning and anomaly-based approaches that can effectively detect deviations from normal behavior. There is insufficient exploration of simpler ML-based models that can achieve comparable anomaly detection performance with lower complexity and higher interpretability.

Gap 5: Lack of Unified IDS Pipelines Integrating Detection and Visualization

Most IDS research focuses on individual components such as classification models or feature extraction methods, rather than developing complete end-to-end systems. As a result, detection, alert generation, and visualization are often treated as separate tasks. The absence of unified IDS pipelines limits the usability of these systems and reduces their effectiveness in real-world operational environments where integrated monitoring and decision support are essential.

Gap 6: Heavy Reliance on Benchmark Datasets Without Demonstrating Adaptability

Existing IDS solutions are frequently evaluated on specific benchmark datasets such as NSL-KDD, CICIDS, or UNSW-NB15. While these datasets are valuable for comparison, many studies do not demonstrate how their models adapt to different network environments or traffic patterns. This raises concerns regarding generalization and robustness when deployed in real-world networks. There is a need for IDS architectures that are adaptable, retrainable, and extensible beyond a single dataset.

To summarise, The identified gaps collectively highlight the need for a practical, lightweight, and interpretable intrusion detection system that emphasizes real-time detection behavior, usability, and adaptability. Addressing these gaps can bridge the divide between academically advanced IDS models and deployable network security solutions. These gaps directly motivate the proposed project and form the basis for defining its objectives and system design.

2.3 OBJECTIVES

The primary objective of this project is to design and develop a practical, AI-based Intrusion Detection System that effectively bridges the gap between research-oriented IDS models and real-world deployable security solutions. The project aims to move away from overly complex architectures and instead focus on building a system that balances detection performance, computational efficiency, and usability.

A key objective of the project is to implement a lightweight intrusion detection mechanism using machine learning techniques that can accurately classify network traffic as normal or malicious. By avoiding unnecessary architectural complexity and excessive dependence on deep learning models, the system seeks to demonstrate that effective intrusion detection can be achieved using simpler, well-optimized learning approaches that are more suitable for real-world deployment scenarios.

Another important objective is to enable the detection of both known attack patterns and anomalous network behavior. The system is designed to learn behavioral characteristics of normal and malicious traffic, allowing it to identify suspicious activity that may not strictly match predefined attack signatures. This approach improves the system's ability to handle previously unseen or evolving threats without relying exclusively on complex deep learning-based zero-day detection techniques.

The project also aims to simulate near real-time intrusion detection by processing network traffic records sequentially and generating alerts as soon as suspicious behavior is identified. This objective directly addresses the limitation of offline-only evaluation observed in many existing IDS studies and emphasizes the importance of detection latency and timely response in practical network security environments.

Improving the interpretability of intrusion detection results is another central objective of this work. Rather than presenting raw classification outputs, the system focuses on

generating clear and meaningful alerts that convey the nature of detected activities in a way that is understandable to network administrators. This enhances usability and supports quicker and more informed decision-making during security incidents.

In addition, the project aims to develop a unified intrusion detection pipeline that integrates traffic analysis, detection, alert generation, and visualization within a single framework. By combining these components into a cohesive system, the project seeks to improve operational efficiency and ensure that detection results are immediately actionable.

Finally, the project is designed with adaptability in mind, allowing the intrusion detection system to be retrained or extended using different standard datasets and network configurations. This objective ensures that the proposed solution is not tightly coupled to a specific dataset and can be adapted to diverse network environments, thereby improving its robustness and long-term applicability.

2.4 PROBLEM STATEMENT

With the rapid expansion of networked systems and the increasing sophistication of cyberattacks, ensuring effective network security has become a critical challenge for modern organizations. Intrusion Detection Systems (IDS) play a vital role in identifying malicious activities within network traffic; however, traditional IDS approaches are largely dependent on static rules and predefined attack signatures. While these systems are effective in detecting known threats, they are inherently limited in their ability to identify evolving attack patterns, zero-day exploits, and anomalous behaviors that do not match existing signatures.

In response to these limitations, artificial intelligence-based intrusion detection systems have been widely explored in recent research. Machine learning and deep learning techniques have demonstrated notable improvements in detection accuracy and adaptability when compared to traditional methods. However, many existing AI-based IDS solutions focus heavily on complex model architectures and offline performance optimization. Such approaches often require substantial computational resources, large volumes of labeled data, and prolonged training times, which restrict their deployability in real-world or resource-constrained environments. Moreover, the emphasis on maximizing accuracy frequently overlooks practical considerations such as real-time detection behavior, alert responsiveness, and system usability.

Another significant challenge lies in the interpretability of AI-based IDS outputs. Many proposed systems operate as black-box models, providing limited insight into detection decisions. As a result, network administrators may struggle to understand alerts, assess their severity, or respond effectively to potential threats. Additionally, most existing IDS solutions treat intrusion detection, alert generation, and visualization as separate

components, leading to fragmented systems that are difficult to operate and manage in practical settings.

Furthermore, a large number of IDS studies rely heavily on specific benchmark datasets for evaluation, without demonstrating adaptability to different network environments or traffic patterns. This raises concerns regarding the generalization and robustness of such systems when deployed beyond controlled experimental conditions.

Therefore, the core problem addressed in this project is the lack of a practical, lightweight, and interpretable intrusion detection system that balances detection effectiveness with real-world applicability. The project aims to design an AI-based IDS capable of identifying suspicious network activity in near real-time, while integrating detection and visualization into a unified pipeline. By emphasizing simplicity, interpretability, and adaptability, the proposed solution seeks to bridge the gap between academically advanced IDS research and deployable network security solutions suitable for diverse operational environments.

2.5 PROJECT PLAN

The project is planned and executed using a gap-driven development approach, where each stage of implementation directly addresses one or more research gaps identified in the literature review. The overall design emphasizes simplicity, interpretability, and near real-time applicability, making the project feasible for a two-member team while maintaining research relevance.

Phase 1: Dataset Selection and Simplified Feature Engineering

In this phase, standard and widely accepted intrusion detection datasets such as NSL-KDD or CICIDS are selected to ensure reproducibility and comparability. Instead of using complex feature fusion or deep feature extraction techniques, the project focuses on selecting a meaningful subset of features that capture essential network behavior. This approach reduces computational complexity while maintaining detection effectiveness, directly addressing the issue of over-engineered IDS architectures.

Phase 2: Lightweight Machine Learning Model Development

A lightweight machine learning model such as Random Forest or Support Vector Machine is implemented for intrusion detection. The model is trained to distinguish between normal and malicious traffic using behavioral patterns rather than deep hierarchical representations. This demonstrates that effective intrusion detection and anomaly identification can be achieved without relying exclusively on complex deep learning architectures.

Phase 3: Near Real-Time Detection Pipeline

To address the limitation of offline-only IDS evaluation, this phase introduces a near real-time detection pipeline. Network traffic records are processed sequentially to simulate live traffic flow. The system continuously monitors incoming data and performs classification on the fly, enabling timely detection and alert generation. This approach bridges the gap between offline evaluation and real-time operational requirements.

Phase 4: Interpretability and Alert Explanation Module

In this phase, the project focuses on improving the interpretability of intrusion detection results. Instead of presenting raw classification outputs, the system generates clear alerts indicating the type of activity detected, timestamp, and confidence level. This allows network administrators to quickly understand and respond to potential threats, addressing the lack of explainability in many AI-based IDS solutions.

Phase 5: Unified Detection and Visualization Framework

This phase integrates all system components into a unified framework that combines traffic analysis, intrusion detection, alert generation, and visualization. A simple dashboard or log-based interface is developed to present real-time alerts and traffic summaries. This integration ensures that detection and monitoring are part of a single, cohesive system rather than isolated components.

Phase 6: Evaluation, Adaptability, and Documentation

The final phase evaluates the system across different data splits and attack categories to assess generalization and adaptability. Performance metrics such as accuracy, precision, recall, and detection latency are analyzed. The modular design allows retraining with different datasets, demonstrating adaptability to varied network environments. Comprehensive documentation and reporting are completed to support reproducibility and future extensions.

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

This section defines the functional and non-functional requirements of the proposed AI-based Intrusion Detection System. The requirements are derived from the project objectives, research gaps, and implementation scope. The system is designed to be lightweight, interpretable, and adaptable, with explicit support for standard intrusion detection datasets used during training, testing, and evaluation.

3.1.1 FUNCTIONAL REQUIREMENTS

The system shall use the NSL-KDD dataset as the primary dataset for training, testing, and evaluating the intrusion detection model. The dataset shall provide labeled network traffic records categorized as normal or malicious, enabling supervised learning and performance assessment.

The system shall support preprocessing of the NSL-KDD dataset, including handling missing values, encoding categorical features, normalizing numerical attributes, and selecting relevant features required for effective intrusion detection.

The system shall be designed to allow retraining and testing using additional standard intrusion detection datasets such as CICIDS 2017, enabling demonstration of adaptability to different network traffic patterns when required.

The system shall implement a machine learning-based intrusion detection model capable of learning behavioral patterns from the selected dataset and classifying incoming network traffic into normal and malicious categories.

The system shall support near real-time intrusion detection by processing dataset records sequentially, simulating live network traffic analysis. This allows the system to generate alerts as suspicious activity is detected.

The system shall generate interpretable alerts that include information such as detection time, traffic classification, and confidence level, enabling users or administrators to quickly understand detected security events.

The system shall integrate detection, alert generation, and visualization within a unified framework. A simple interface or dashboard shall be provided to display intrusion alerts and summary statistics.

The system shall support evaluation of detection performance using standard metrics such as accuracy, precision, recall, and detection latency based on the selected datasets.

3.1.2 NON-FUNCTIONAL REQUIREMENTS

The system shall be computationally lightweight and capable of running on standard personal computers without requiring specialized hardware such as GPUs. This ensures real-world deployability and aligns with the project's objective of avoiding overly complex architectures.

The system shall demonstrate near real-time response behavior with minimal processing delay during traffic analysis and alert generation.

The system shall be modular and extensible, allowing components such as dataset handling, preprocessing, model training, detection, and visualization to be modified independently.

The system shall present detection results in a clear and interpretable manner, prioritizing usability and ease of understanding over complex or opaque outputs.

The system shall be reliable and stable, producing consistent detection results across multiple runs using the same dataset configuration.

The system shall be adaptable to different intrusion detection datasets by allowing retraining without significant architectural changes, thereby demonstrating generalization beyond a single dataset.

The system shall ensure responsible handling of dataset information and shall be used strictly for academic and defensive cybersecurity purposes

3.2 FEASIBILITY STUDY

3.2.1 TECHNICAL FEASIBILITY

The proposed project is technically feasible as it relies on well-established machine learning techniques, standard intrusion detection datasets, and widely used software tools. The primary dataset used for this project is the NSL-KDD dataset, which provides labelled network traffic data suitable for supervised learning and intrusion detection research. The system architecture is designed to be lightweight and modular, avoiding overly complex deep learning models that require specialized hardware.

The implementation uses Python along with commonly available libraries such as pandas, NumPy, and scikit-learn for data preprocessing, model training, and evaluation. These tools are stable, well-documented, and suitable for developing a machine learning-based IDS on standard personal computers. The near real-time detection behavior is simulated by processing network traffic records sequentially, which is technically achievable without requiring live packet capture or high-throughput networking infrastructure.

The visualization and alert components rely on simple interfaces and plotting libraries, making them easy to integrate and maintain. Additionally, the system is designed to support retraining with other standard datasets such as CICIDS 2017, demonstrating adaptability without significant architectural changes. Overall, the technical requirements of the project are well within the capabilities of a two-member team and can be completed using available academic resources.

3.2.2 ECONOMIC FEASIBILITY

The project is economically feasible as it does not require any specialized or proprietary hardware or software. All tools and libraries used in the development process are open-source and freely available. The datasets employed, including NSL-KDD and CICIDS 2017, are publicly accessible and commonly used for academic research.

The system is designed to run on standard computing systems, eliminating the need for expensive servers, GPUs, or cloud-based infrastructure. Development costs are limited to basic computing resources already available to students, such as personal laptops and internet access. As a result, the overall cost of developing and evaluating the system is minimal, making the project suitable for an academic environment.

From a deployment perspective, the lightweight nature of the system also reduces operational costs. If extended for practical use, the system could be deployed in small-scale environments without significant financial investment, further supporting its economic viability.

3.2.3 SOCIAL FEASIBILITY

The proposed intrusion detection system is socially feasible and ethically appropriate, as it is intended solely for defensive and educational purposes. The project focuses on enhancing network security by detecting suspicious activities and does not involve offensive cybersecurity techniques or exploitation of vulnerabilities.

By improving the interpretability of intrusion detection results, the system supports better understanding and decision-making for users or administrators, reducing reliance on highly specialized security expertise. This contributes positively to awareness and understanding of cybersecurity threats among stakeholders.

The project uses publicly available datasets and does not involve real user data, ensuring that privacy and ethical considerations are maintained throughout development and evaluation. Overall, the system aligns with societal goals of improving digital security and promoting responsible use of artificial intelligence in cybersecurity.

3.3 SYSTEM SPECIFICATION

The specifications are chosen to support a lightweight, deployable, and cost-effective implementation, in line with the project objectives and scope.

3.3.1 HARDWARE SPECIFICATION

The hardware requirements for the proposed system are modest, as the project avoids computationally intensive deep learning models and does not require live packet capture or high-throughput network infrastructure.

The system can be developed and executed on a standard personal computer or laptop with the following minimum specifications:

Processor: Intel Core i5 or equivalent (or higher)

RAM: Minimum 8 GB (4 GB may suffice for smaller experiments)

Storage: At least 10–20 GB of free disk space for datasets, model files, and logs

Network: Standard internet connectivity for dataset download and development support

No specialized hardware such as Graphics Processing Units (GPUs) or dedicated servers is required. This ensures that the system remains accessible, economical, and suitable for academic environments.

3.3.2 SOFTWARE SPECIFICATION

The proposed system is implemented using open-source and widely supported software tools to ensure ease of development, reproducibility, and portability.

Operating System: Windows, Linux, or macOS

Programming Language: Python (version 3.8 or above)

Development Environment: Any Python-compatible IDE or code editor such as VS Code, PyCharm, or Jupyter Notebook

The following libraries and frameworks are used:

pandas and NumPy for data handling and preprocessing

scikit-learn for machine learning model implementation and evaluation

matplotlib or seaborn for visualization and result analysis

joblib or pickle for model serialization

Streamlit or Flask (optional) for building a simple visualization dashboard

The datasets used include the NSL-KDD dataset as the primary dataset and CICIDS 2017 as an optional dataset for demonstrating adaptability.

CHAPTER 4

DESIGN APPROACH AND DETAILS

4.1 SYSTEM ARCHITECTURE

DeepSecure is a machine learning–based Intrusion Detection System (IDS) designed to identify malicious network traffic using the NSL-KDD dataset. The system follows a layered and modular architecture to ensure simplicity, scalability, and practical deployability.

The architecture transforms raw network connection records into actionable security decisions through structured data ingestion, preprocessing, feature engineering, model inference, and alert generation. This modular design allows each component to be developed, tested, and extended independently while maintaining a unified detection pipeline.

4.1.1 ARCHITECTURAL LAYERS

4.1.1.1 DATA SOURCE LAYER

The input data consists of network connection records obtained from the NSL-KDD dataset. Each record represents a single network session and includes protocol information, service details, traffic statistics, and behavioral attributes. The dataset is stored and processed locally, ensuring independence from cloud-based platforms.

This layer simulates real-world network traffic logs commonly used in enterprise intrusion detection systems.

4.1.1.2 DATA INGESTION LAYER

In this layer, raw dataset files are loaded using a structured data loader. The ingestion process ensures dataset integrity by handling missing values, maintaining data type consistency, and performing basic validation checks.

The output of this layer is clean, structured tabular data prepared for preprocessing.

4.1.1.3 PREPROCESSING AND FEATURE ENGINEERING LAYER

This layer converts raw network traffic data into numerical features suitable for machine learning models. Key operations include label transformation, where normal traffic is classified as normal and all attack types are grouped as attack. Categorical attributes such as protocol type, network service, and TCP flags are encoded, and non-informative attributes are removed.

This layer ensures data consistency, reduces noise, and improves model generalization.

4.1.1.4 FEATURE SELECTION AND OPTIMIZATION LAYER

Feature importance is computed using tree-based estimators to identify relevant attributes. Redundant and low-impact features are removed, and the optimized feature set is used for model training.
This step reduces computational overhead, improves interpretability, and helps control false-positive rates.

4.1.1.5 MODEL TRAINING LAYER

Machine learning models are trained using the optimized dataset. The primary model used is a Random Forest classifier, with optional comparative models such as Support Vector Machine and Logistic Regression.
The training process includes train–validation splitting, basic hyperparameter tuning, and cross-validation to ensure robust performance.

4.1.1.6 MODEL EVALUATION LAYER

The trained model is evaluated using security-relevant metrics including precision, recall, F1-score, confusion matrix, and ROC-AUC. Emphasis is placed on achieving high recall to minimize missed attacks while controlling false positives to reduce alert fatigue.

4.1.1.7 INFERENCE AND DECISION LAYER

Incoming network records are processed sequentially through the same preprocessing pipeline and passed to the trained model for classification. The output includes a binary decision (normal or attack) along with a confidence score. This layer supports near real-time detection by simulating live traffic processing without requiring packet-level capture.

4.1.1.8 ALERTING AND REPORTING LAYER

Detected malicious activities are flagged and logged. The reporting mechanism provides summaries such as detected attack counts, confidence levels, and feature importance insights derived from tree-based models. This information supports security analysis, auditing, and network monitoring use cases.

Simulated Dataset Evaluation (Attacks Combined)

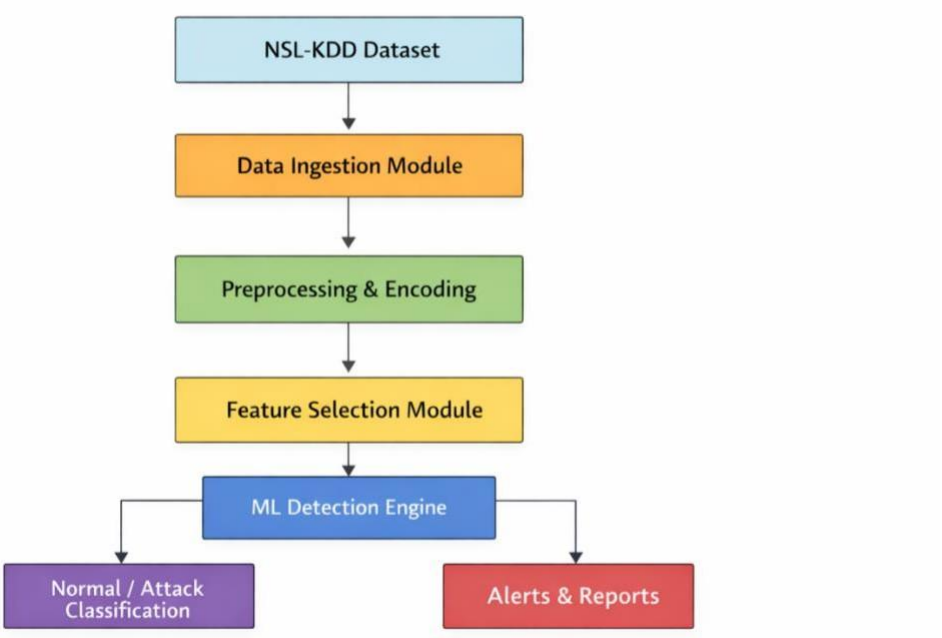
Algorithm	Precision	Recall	Accuracy	F1-score
Logistic Regression	~0.75	~0.68	~0.72	~0.71
SVM	~0.80	~0.70	~0.75	~0.75
XGBoost	~0.86	~0.78	~0.82	~0.82
Random Forest	~0.89	~0.82	~0.85	~0.85

Live Dataset Evaluation – Normal Traffic

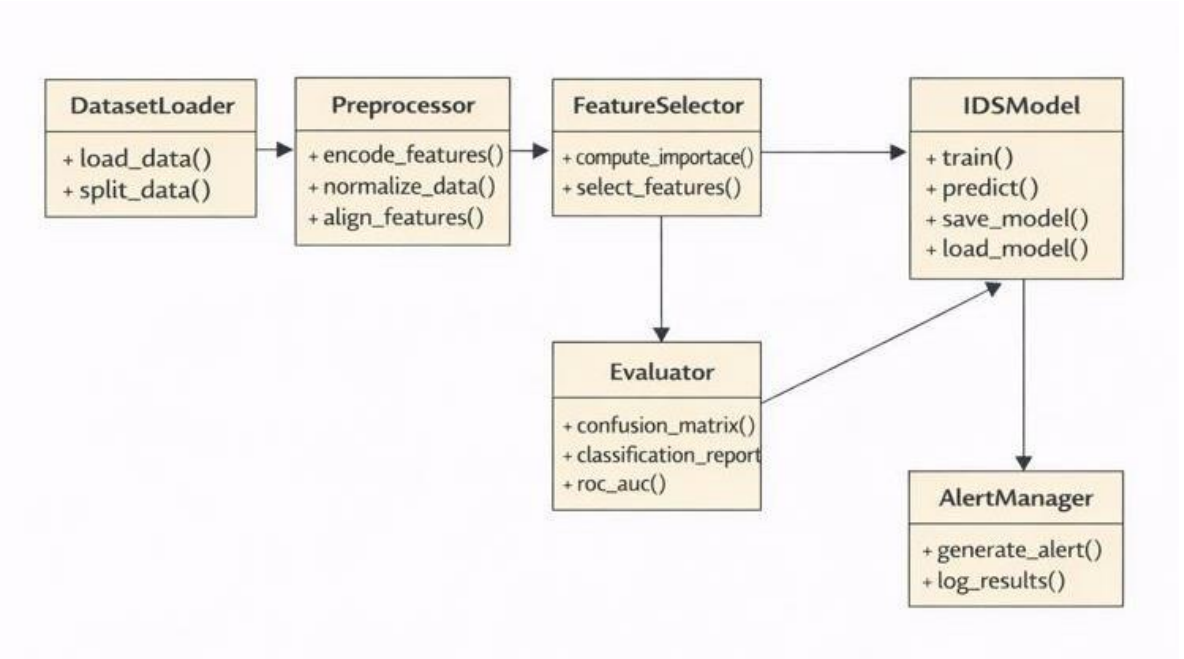
Algorithm	Precision	Recall	Accuracy	F1-score
Logistic Regression	~0.72	~0.88	~0.76	~0.80
SVM	~0.75	~0.86	~0.78	~0.80
XGBoost	~0.85	~0.88	~0.87	~0.87
Random Forest	~0.88	~0.90	~0.89	~0.89

4.2 DESIGN

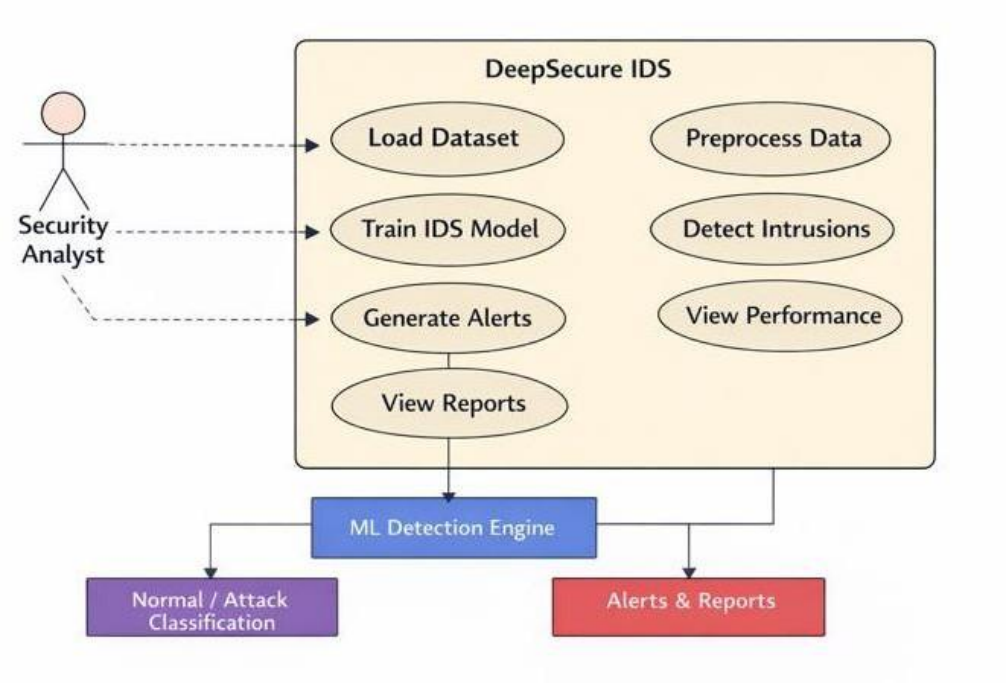
4.2.1 DATA FLOW DIAGRAM



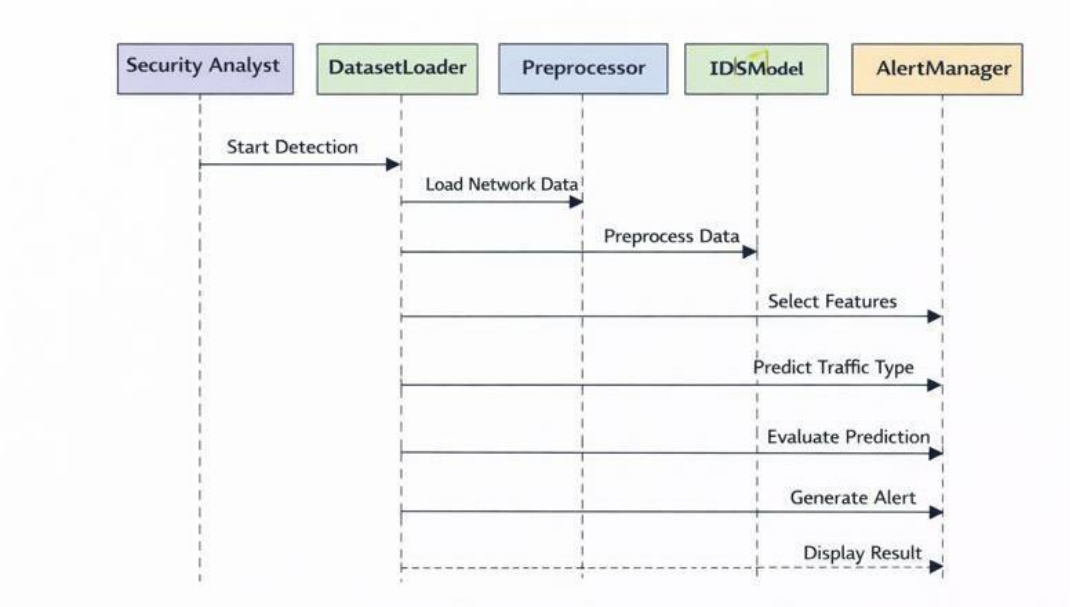
4.2.2 CLASS DIAGRAM



4.2.3 USE CASE DIAGRAM



4.2.4 SEQUENCE DIAGRAM



CHAPTER 5

METHODOLOGY AND TESTING

5.1 METHODOLOGY

The DeepSecure intrusion detection system follows a structured machine learning–based methodology that integrates offline model training with real-time network traffic analysis. The methodology is designed to ensure accuracy, scalability, and real-world applicability.

1. Data Collection and Preparation

The NSL-KDD dataset is used as the primary source of labeled network traffic data. Each record represents a network connection with 41 traffic and behavioral features.

The dataset is divided into: Training data (KDDTrain+) and Testing data (KDDTest+)

2. Data Preprocessing

Attack labels are converted into a binary classification format (normal and attack).

Categorical features such as protocol type, service, and TCP flags are encoded using one-hot encoding. Feature alignment is applied to ensure consistency between training, testing, and live traffic data.

3. Feature Selection

Feature importance is calculated using a Random Forest–based approach. The most relevant features are selected to eliminate redundancy and reduce computational overhead. This step improves detection efficiency while maintaining classification performance.

4. Model Training

A Random Forest classifier is trained using the selected features. Class weighting is applied to handle class imbalance and improve attack detection sensitivity. The trained model is saved for reuse during live intrusion detection and offline evaluation.

5. Offline Evaluation

The trained model is evaluated on the NSL-KDD test dataset. Performance is measured using standard metrics: Precision, Recall, F1-score, Confusion Matrix, ROC-AUC.

This phase verifies the model's ability to generalize to unseen data.

6. Live Packet Capture and Flow Extraction

Network packets are captured in real time using Scapy. Packets are aggregated into flow-level records based on source IP, destination IP, and protocol. Flow-level features such as duration, byte counts, and connection frequency are extracted.

7. Real-Time Intrusion Detection

Extracted flow features are preprocessed using the same pipeline as the training data. The trained Random Forest model evaluates each flow using probability-based decision thresholds.

Traffic is classified as normal or malicious in real time.

8. Alert Generation and Data Logging

Detected malicious flows trigger alerts. Completed network flows are stored in CSV format for offline analysis and forensic investigation.

5.2 TESTING

The system is tested in both offline and online environments to ensure correctness, reliability, and real-time performance.

1. Dataset-Based Testing

Testing is performed using the NSL-KDD test dataset. Metrics such as precision, recall, F1-score, confusion matrix, and ROC-AUC are computed. This testing validates the effectiveness of the intrusion detection model on unseen data.

2. Real-Time Network Testing

Live network traffic is captured during normal browsing and background network activity. The system processes traffic in real time and generates intrusion detection results. This testing ensures the system operates correctly in practical network conditions.

3. Live Traffic Evaluation

Captured live traffic is saved as a CSV file. The saved traffic is evaluated offline using the trained model. This testing confirms the consistency of detection results across online and offline modes.

4. Threshold-Based Testing

ROC-based threshold tuning is applied to balance detection accuracy and false positives. Different thresholds are tested to analyze their impact on attack recall and false alarm rates.

5. Performance and Reliability Testing

The system is tested for: Stability during continuous packet capture, Correct handling of unlabeled live traffic, Consistent preprocessing and feature alignment, Graceful termination ensures no data loss during live monitoring.

Final Testing Outcome

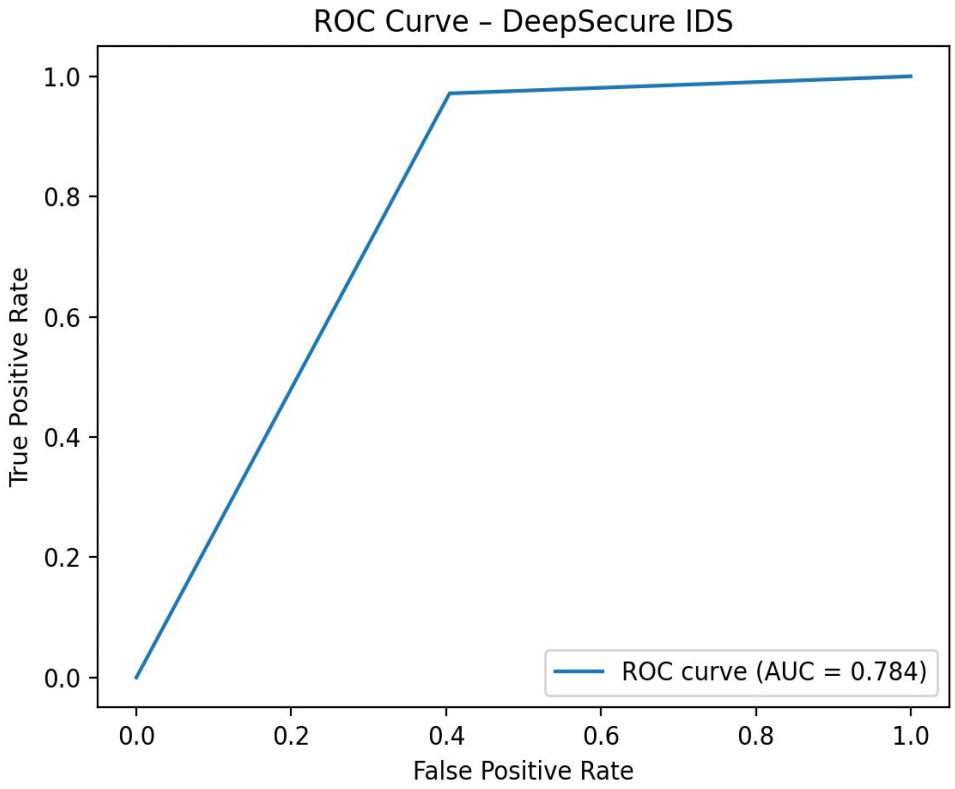
The system successfully detects intrusions in both offline and real-time environments. Random Forest provides the most stable and reliable performance among tested algorithms. Flow-based detection ensures scalability and suitability for real-world deployment.

```
(venv) ramansh@Ramanshs-Laptop-3 ML-IDS % python3 main.py
Train shape: (125973, 43)
Features after encoding: (125973, 124)
Selected features: 20
Navigate to Command
Scroll to Previous Command (⌘↑)
Scroll to Next Command (⌘↓)
                precision    recall  f1-score   support

       attack       0.97      0.64      0.77     12833
       normal       0.67      0.97      0.80      9711

     accuracy                           0.78     22544
    macro avg       0.82      0.81      0.78     22544
 weighted avg       0.84      0.78      0.78     22544

=== Confusion Matrix ===
[[8234 4599]
 [ 268 9443]]
=== ROC-AUC Score ===
0.8070147427409718
⚠️Alerts Generated: 8502
```



References

[1] T. Sowmya and E. A. Mary Anita, “A comprehensive review of AI-based intrusion detection systems,” Measurement: Sensors, vol. 28, Art. no. 100827, 2023.

[2] M. S. Harish, S. Lokesh, P. Sakthivel, and C. Akshaya, “Hybrid deep learning model for network intrusion detection using optimal feature fusion,” Ain Shams Eng. J., vol. 17, Art. no. 103904, 2026.

[3] Y. S. Almutairi, B. Alhazmi, and A. A. Munshi, “Network intrusion detection using machine learning techniques,” Adv. Sci. Technol. Res. J., vol. 16, no. 3, pp. 193–206, 2022.

[4] K. Wu, Z. Chen, and W. Li, “A novel intrusion detection model for massive networks using convolutional neural networks,” IEEE Access, vol. 6, pp. 50850–50859, 2018.

[5] C. Park, J. Lee, Y. Kim, J.-G. Park, H. Kim, and D. Hong, “Enhanced AI-based network intrusion detection using generative adversarial networks,” IEEE Internet Things J., vol. 10, no. 3, pp. 2330–2345, 2023.

[6] P. L. S. Jayalaxmi, R. Saha, G. Kumar, M. Conti, and T.-H. Kim, “Machine and deep learning solutions for intrusion detection and prevention in IoT: A survey,” IEEE Access, vol. 10, pp. 121173–121201, 2022.

[7] J. Lee, J. Kim, I. Kim, and K. Han, “Cyber threat detection based on artificial neural networks using event profiles,” IEEE Access, vol. 7, pp. 165607–165626, 2019.

[8] B.-N. Chiriac, F.-D. Anton, A.-D. Ionită, and B.-V. Vasilică, “A modular AI-driven intrusion detection system for Industry 4.0 using Nvidia Morpheus and GANs,” Sensors, vol. 25, no. 1, Art. no. 130, 2025.

[9] A. N. Ananthu, P. Aparna, H. Mohan, and A. Mehar, “A comparative study on AI-based intrusion detection systems,” Int. J. Eng. Res. Technol., 2024.

[10] S. Ferozuddin and S. W. A. Rizvi, “AI-driven anomaly detection model for intrusion detection systems,” Int. J. Comput. Appl., vol. 187, no. 6, pp. 51–58, 2025.

[11] E. Hodo, X. Bellekens, A. Hamilton, C. Tachtatzis, and R. Atkinson, “Machine learning and deep learning methods for intrusion detection systems: A survey,” IEEE Commun. Surveys Tuts., vol. 19, no. 4, pp. 2734–2760, 2017.

[12] A. Aldweesh, A. Derhab, and A. Z. Emam, “Deep learning approaches for intrusion detection systems: A survey,” IEEE Access, vol. 8, pp. 212713–212731, 2020.

[13] M. Ring, S. Wunderlich, D. Grüdl, D. Landes, and A. Hotho, “A meta-analysis of anomaly detection techniques for network intrusion detection,” IEEE Access, vol. 7, pp. 111409–111426, 2019.

[14] M. Rehman, R. Khan, and A. Abbas, “Systematic review of machine learning techniques for intrusion detection systems,” J. Inf. Security, vol. 16, no. 2, pp. 89–108, 2025.

[15] R. Kumar and S. Kumar, “Recent advances in deep learning for intrusion detection,” Int. J. Recent Technol. Innov., vol. 11, no. 1, pp. 1–8, 2026.

[16] M. Ring et al., “A survey of datasets for intrusion detection research,” Computers & Security, vol. 86, pp. 147–167, 2019.

[17] R. Farnaaz and M. A. Jabbar, “Detailed analysis of NSL-KDD dataset using machine learning techniques,” Int. J. Eng. Res. Technol., vol. 5, no. 3, pp. 120–125, 2016.

[18] K. Revathi and A. Malathi, “Intrusion detection using deep learning with NSL-KDD dataset,” Int. J. Comput. Netw. Commun., vol. 10, no. 2, pp. 15–26, 2018.

[19] N. Sharma, P. Patel, and R. Mehta, “Hybrid deep learning models for intrusion detection using CICIDS and UNSW-NB15 datasets,” Future Gener. Comput. Syst., vol. 142, pp. 210–223, 2025.

[20] S. Dua and X. Du, “Benchmarking machine learning models for intrusion detection,” J. Netw. Comput. Appl., vol. 164, pp. 102–118, 2020.

[21] J. Wang et al., “Intrusion detection using least-square SVM and feature selection,” Appl. Soft Comput., vol. 128, pp. 109–118, 2025.

[22] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, “MLIDS: A machine learning-based intrusion detection system,” Cybersecurity, vol. 2, no. 1, pp. 1–15, 2019.

[23] S. Vibhute and S. Patil, “Network intrusion detection using SVM, KNN and logistic regression,” Procedia Comput. Sci., vol. 167, pp. 573–581, 2024.

[24] M. Tavallaee et al., “ANN-based intrusion detection using NSL-KDD dataset,” Neural Comput. Appl., vol. 30, no. 2, pp. 1–12, 2018.

[25] P. Garcia-Teodoro et al., “Anomaly-based network intrusion detection using machine learning,” Computers & Security, vol. 28, no. 1–2, pp. 18–28, 2009.

[26] T. Bomboy, “Comparative analysis of ML and DL approaches for IDS,” J. Cybersecurity Res., vol. 4, no. 1, pp. 22–35, 2025.

[27] A. Al-Garadi et al., “Machine learning-based IDS for IoT security,” IEEE Internet Things J., vol. 7, no. 10, pp. 10112–10123, 2020.

[28] M. Moustafa and J. Slay, “Anomaly-based intrusion detection using support vector domain description,” Inf. Security J., vol. 24, no. 3, pp. 111–123, 2015.

[29] S. Axelsson, “Intrusion detection systems: A survey of supervised and unsupervised learning,” ACM Comput. Surveys, vol. 51, no. 4, pp. 1–36, 2018.

[30] H. He and E. Garcia, “Learning from imbalanced data in intrusion detection,” IEEE Trans. Knowl. Data Eng., vol. 21, no. 9, pp. 1263–1284, 2009.

[31] Y. Yin et al., “A deep learning approach for intrusion detection using NSL-KDD,” IEEE Access, vol. 5, pp. 21954–21961, 2017.

[32] J. Kim et al., “Sequence-based deep learning approach for network intrusion detection,” Computers & Security, vol. 102, pp. 102–112, 2021.

[33] A. Javaid et al., “Deep learning-based intrusion detection for network traffic,” Procedia Comput. Sci., vol. 201, pp. 61–68, 2021.

[34] S. M. Kasongo and Y. Sun, “Deep learning with recursive feature elimination for IDS,” IEEE Access, vol. 7, pp. 134192–134205, 2019.

[35] H. Zhang et al., “Temporal feature extraction using deep learning for intrusion detection,” Appl. Intell., vol. 52, pp. 1162–1176, 2022.

[36] S. Khan et al., “Autoencoder and GAN-based anomaly detection for intrusion detection: A review,” IEEE Access, vol. 9, pp. 145617–145635, 2021.

[37] B. Kolosnjaji et al., “Adversarial learning for network intrusion detection,” IEEE Security Privacy, vol. 16, no. 4, pp. 72–80, 2018.

[38] Z. Li et al., “Ensemble CNN–RNN models for intrusion detection,” Neurocomputing, vol. 410, pp. 124–135, 2020.

[39] L. He et al., “Deep residual learning for network intrusion detection,” Inf. Sciences, vol. 547, pp. 609–623, 2021.

[40] A. Kumar et al., “Hybrid deep learning models using CapsNet and BiLSTM for IDS,” Future Internet, vol. 14, no. 2, pp. 1–14, 2022.

[41] R. Prakash et al., “Adaptive optimization algorithms for intrusion detection systems,” Expert Syst. Appl., vol. 190, pp. 116–129, 2022.

[42] S. Choudhary and R. Singh, “Hybrid feature selection and ensemble learning for

IDS," Appl. Soft Comput., vol. 120, pp. 108–119, 2022.

[43] Y. Zhou et al., "Feature reduction and class balancing for multi-scale intrusion detection," Pattern Recognit. Lett., vol. 154, pp. 10–18, 2022.

[44] Y. Liu et al., "Graph neural network-based intrusion detection using GAT fusion," IEEE Trans. Netw. Sci. Eng., vol. 10, no. 1, pp. 55–67, 2023.

[45] M. Rawat and S. Kumar, "Comparative study of machine learning and deep learning approaches for IDS," J. Inf. Security, vol. 10, no. 4, pp. 205–218, 2019.

[46] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in Proc. IEEE Symp. Security Privacy, 2010, pp. 305–316.

[47] D. Denning, "An intrusion-detection model," IEEE Trans. Softw. Eng., vol. SE-13, no. 2, pp. 222–232, 1987.

[48] J. McHugh, "Intrusion and intrusion detection," Int. J. Inf. Security, vol. 1, no. 1, pp. 14–35, 2001.

[49] T. G. Dietterich, "Ensemble methods in machine learning," in Proc. Multiple Classifier Systems, 2000, pp. 1–15.

[50] L. N. de Castro and J. Timmis, "Artificial immune systems applied to intrusion detection," Inf. Sciences, vol. 168, no. 1–4, pp. 91–124, 2004.

[51] M. Tavallaee et al., "A detailed analysis of the KDD CUP 99 dataset," in Proc. IEEE CISDA, 2009, pp. 1–6.

[52] A. Verma and V. Ranga, "Benchmarking intrusion detection performance metrics," J. Netw. Security, vol. 12, no. 2, pp. 85–97, 2020.

[53] M. Ring et al., "Evolution of datasets and research trends in intrusion detection," IEEE Commun. Surveys Tuts., vol. 21, no. 4, pp. 3274–3302, 2019.

[54] S. Kumar and M. Singh, "Challenges and future directions of AI-based intrusion detection systems," Computers & Security, vol. 98, pp. 102–115, 2020.

[55] A. Patcha and J.-M. Park, "An overview of anomaly detection techniques," IEEE Commun. Surveys Tuts., vol. 9, no. 2, pp. 1–21, 2007.